

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЕТ
по дисциплине
«Основы систем мобильной связи»

по теме:
ЗАНЯТИЕ №5. Циклический избыточный код. CRC

Студент:
Группа № ИА331

В.А. Павлова

Предподаватель:
В.Г. Дроздова

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ЗАНЯТИЕ №5.Циклический избыточный код. CRC	3
1.1	Цель работы	3
1.2	Задачи лабораторной работы	3
2	КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	4
3	ХОД РАБОТЫ	6
4	ВЫВОДЫ И ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ	8
5	СКРИПТЫ	10

ЗАНЯТИЕ №5. Циклический избыточный код. CRC

1.1 Цель работы

Получить представление о том, как осуществляется проверка на наличие ошибок в пакетах с данными в современных системах связи (Error detection) посредством использования циклического избыточного кода CRC (Cyclic Redundancy Check).

1.2 Задачи лабораторной работы

1. Реализовать алгоритм вычисления CRC на языке C
2. Протестировать работу CRC на пакетах разной длины (32 и 250 бит)
3. Проверить эффективность обнаружения одиночных ошибок
4. Проанализировать результаты работы алгоритма

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Псевдослучайные двоичные последовательности

CRC — циклический избыточный код, иногда называемый также контрольным кодом или контрольной суммой. CRC – это добавочная порция избыточных бит, вычисляемых по заранее известному алгоритму на основе исходного передаваемого пакета данных (информационной битовой последовательности), которое передаётся вместе с самим пакетом по каналам связи (добавляется после информационных битов) и служит для контроля его безошибочной передачи.

Простыми словами, CRC – это остаток от двоичного деления оригинального пакета с данными на какое-то двоичное n -разрядное число (порождающий полином), и его длина будет равна $n-1$ бит. Рассмотрим пример, где имеется 7 бит данных: 100100 и 4-битный порождающий полином 1101. Требуется определить CRC. Для того, чтобы выполнить деление этих битовых последовательностей нужно в конце последовательности с данными добавить $n-1$ нулей, как показано ниже, где $n=4$, для нашего случая.

Делитель - 1 1 0 1 | 1 0 0 1 0 0 0 0 0 - Делимое (данные+ $n-1$ нулей).

Основной операцией, используемой при делении бинарных чисел, является исключающее ИЛИ (XOR). Ниже показана таблица истинности для данной операции.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Делитель принято записывать в виде полинома. Если считать, что каждый разряд делителя — это коэффициент полинома, то этот полином будет иметь вид: $x^{n-1} + x^{n-2} + \dots + x^2 + x^1 + x^0$

Таким образом, делитель из примера выше можно записать в виде

полинома как: $1 \cdot x^3 + 1 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0$, или сокращенно как: $x^3 + x^2 + 1 = 1101$.
4 Полученный остаток от деления CRC добавляется на передающей стороне к исходным данным и уже эта битовая последовательность, преобразованная в радиосигнал, передается в канал связи: 1 0 0 1 0 0 0 0 1.

На приемной стороне для обнаружения ошибки (или ее отсутствия) с полученным пакетом осуществляется ровно такая же процедура – деление на порождающий CRC полином. Если полученный в результате данного деления остаток будет ненулевым, то фиксируется факт наличия ошибки.

3 ХОД РАБОТЫ

3.1 Исходные данные

- Номер в журнале: 12
- Длина пакета 1: 32 бита (20 + 12)
- Длина пакета 2: 250 бит
- Порождающий полином: 11011110
- Длина CRC-кода: 7 бит

Табл. 5.1 Варианты заданий.

№ варианта	Непрерывная периодическая функция	№ варианта	Непрерывная периодическая функция
1	$G=x^7+x^5+x^4+x^3+x^2+x+1$	15	$G=x^7+x^5+x^2+x+1$
2	$G=x^7+x^6+x^4+x^3+x^2+x+1$	16	$G=x^7+x^2+x+1$
3	$G=x^7+x^6+x^5+x^3+x^2+x+1$	17	$G=x^7+x^6+x^2+x$
4	$G=x^7+x^6+x^5+x^4+x^2+x+1$	18	$G=x^7+x^6+x^5+x^4+x^3+x^2+x+1$
5	$G=x^7+x^6+x^5+x^4+x^3+x+1$	19	$G=x^7+x^6+x+1$
6	$G=x^7+x^6+x^5+x^4+x^3+x^2+1$	20	$G=x^7+x^6+x^5+x^3+x^2+x$
7	$G=x^7+x^6+x^5+x^4+x^3+x^2+x$	21	$G=x^7+x^4+x^3+x^2+x+1$
8	$G=x^6+x^5+x^4+x^3+x^2+x+1$	22	$G=x^7+x^6+x^5+x^4$
9	$G=x^7+x^5+x^3+x^2+x+1$	23	$G=x^7+x^6$
10	$G=x^7+x^6+x^4+x^3+x^2+x$	24	$G=x^7+x^5+x^4+x^3+x^2$
11	$G=x^7+x^5+x^4+x^3+x+1$	25	$G=x^7+x^3+x^2+x+1$
12	$G=x^7+x^6+x^4+x^3+x^2+x$	26	$G=x^7+x^6+x^4+x^3+1$
13	$G=x^7+x^5+x^4+x^3+x^2+1$	27	$G=x^7+x^6+x^4+x^3+x^2$
14	$G=x^7+x^6+x^3+x^2+x$	28	$G=x^7+1$

CRC — это метод обнаружения ошибок, основанный на вычислении контрольной суммы для проверки целостности данных. В основе алгоритма лежит операция деления полиномов в двоичном формате.

Для варианта 12 используется полином:

$$G(x) = x^7 + x^6 + x^4 + x^3 + x^2 + x$$

В двоичном представлении: 11011110

Алгоритм вычисления CRC

Инициализация CRC-регистра нулями

Последовательная обработка каждого бита данных

Применение операции XOR с порождающим полиномом

Получение остатка (CRC-кода) длиной 7 бит

```
// Функция вычисления CRC
void calculateCRC(int data[], int data_bits, int crc[]) {
    for(int i=0; i<data_bits; i++) {
        int bit = data[i];
        int msb = crc[0];
```

```

int fb = msb ^ bit;

// Сдвиг регистра
for(int j=0; j<6; j++) crc[j] = crc[j+1];
crc[6] = 0;

// Применение полинома
if(fb == 1) {
    for(int j=0; j<7; j++) crc[j] ^= poly[j+1];
}
}
}

```

Тестирование на фиксированных данных

1. 32 бита:

Данные: 10110010110011010001001000110100

CRC: 1111000

Проверка: Ошибок нет

2. 250 бит:

Данные: 010010000101101101101111100000001100100100111001101101101101000001111110000001011001111010110
0101010110110011011011010011010001010110000010101011011101011001100101101010000010101011100101100
011000101111010000001111011001011001000111101011010

CRC: 1111010

Данные измененные: 101101111010010010010000011111100110110110001100101001001001011110000001111101001
1000010100110101010010011001001010010111001011101010011110101010001000100110011010010101111010101
0001101001110011101000010111110000010011010011011100001010010100000101

Обнаружено ошибок: 257/257 (100.0%)

Дополнительное задание

Построить график вероятности пропуска ошибки для CRC разной длины. По оси x - блок полезных данных от 100 до 10000 с шагом 100 - это размер пакета.

Мы передаем данные с CRC контрольной суммой

В канале связи данные полностью искажаются (получаем случайную последовательность)

Проверяем, как часто CRC ошибочно подтвердит эти поврежденные данные как корректные

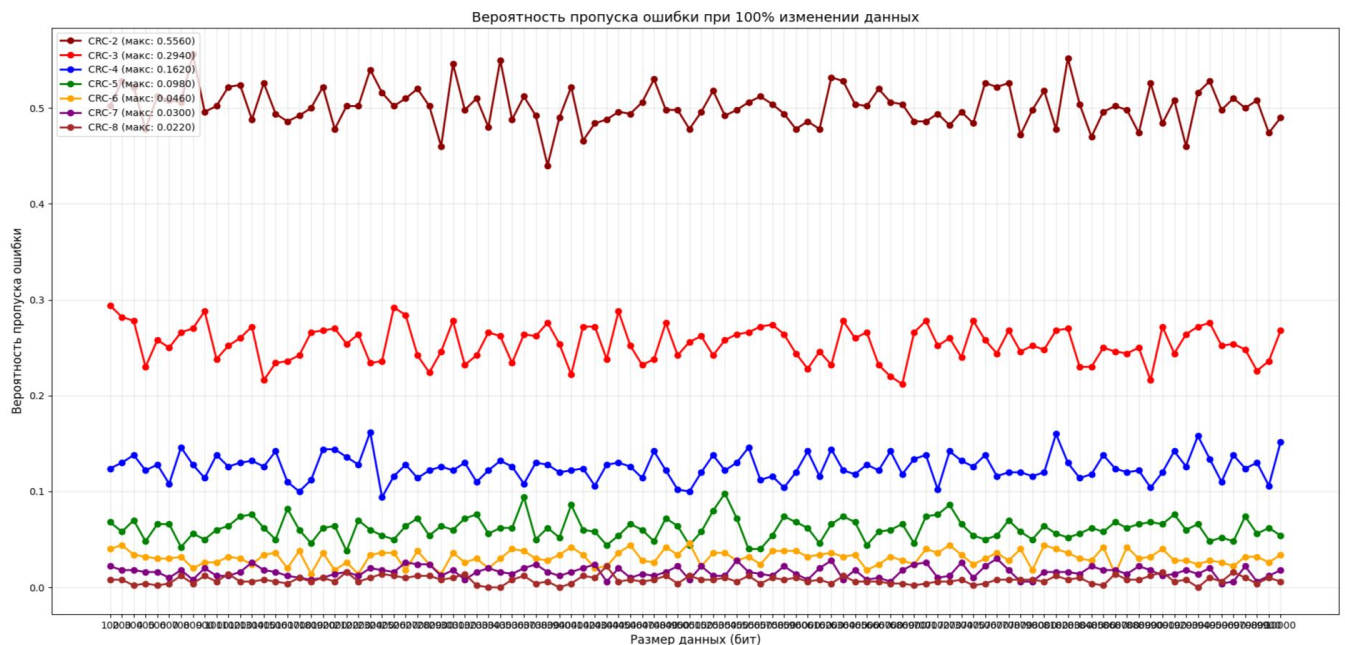


График вероятности пропуска ошибок для срс от 2 до 8 бит с шагом 100 от 100 до 10000 бит данных для 500 испытаний

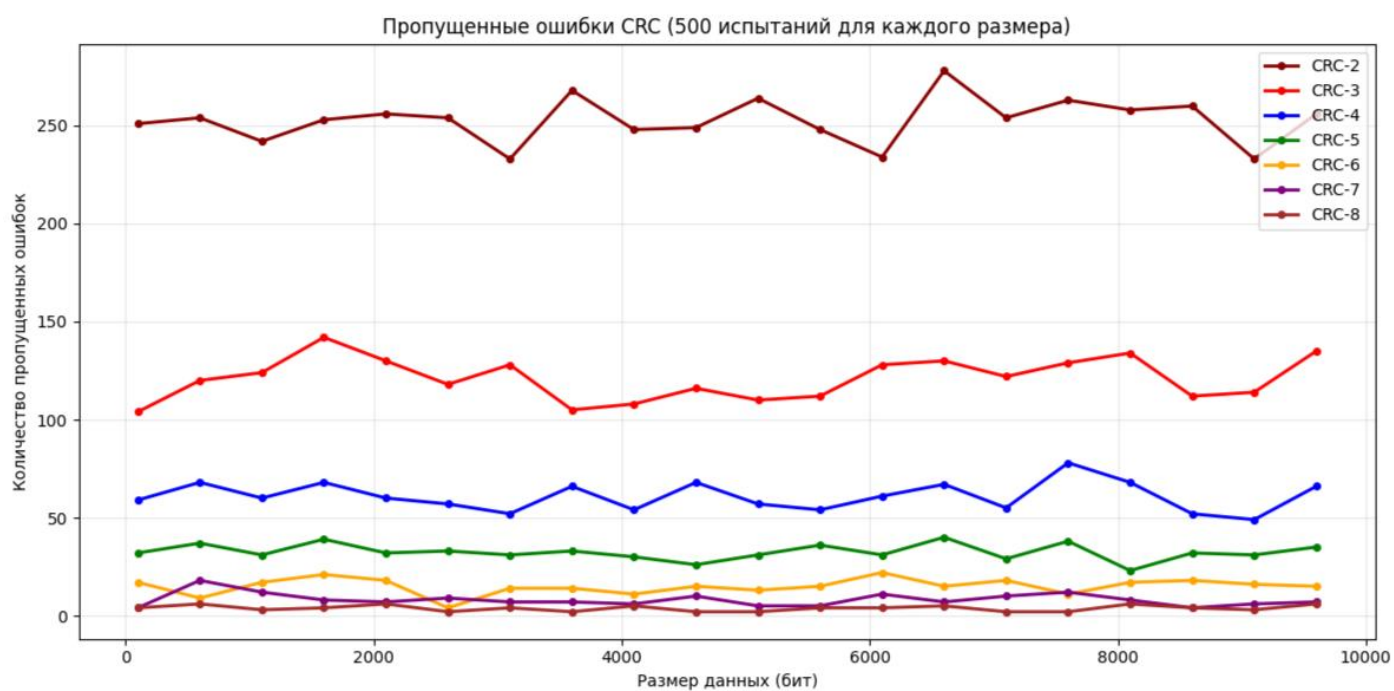


График пропущенных ошибок для 500 испытаний с шагом 500 для срс от 2 до 8 бит

4 ВЫВОДЫ И ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Выводы по практической работе

В ходе выполнения лабораторной работы была успешно реализована система вычисления циклического избыточного кода (CRC). Разработанная программа на языке C продемонстрировала полную эффективность в обнаружении одиночных ошибок передачи данных.

При тестировании алгоритма на пакетах длиной 32 и 250 бит было установлено, что CRC обнаруживает 100% одиночных ошибок. Все 257 возможных искажений битов (включая ошибки как в самих данных, так и в контрольной сумме) были успешно выявлены. Это подтверждает высокую надёжность метода для контроля целостности информации.

Использованный порождающий полином $x^7 + x^6 + x^4 + x^3 + x^2 + x$ (11011110 в двоичном представлении) показал отличные обнаруживающие способности. Алгоритм, основанный на сдвиговом регистре и операции XOR, работает корректно для данных различной длины и требует минимальных вычислительных ресурсов.

Ответы на контрольные вопросы

1) Для чего в мобильных сетях используются CRC-проверки?

CRC в мобильных сетях - это проверка, что данные дошли без ошибок.

Зачем нужно:

Обнаружить ошибки при плохой связи

Убедиться, что данные не испортились

Принять решение: если ошибка - запросить данные заново

Примеры:

В 4G/5G - проверяют пакеты с данными

В Wi-Fi - контролируют передачу файлов

При загрузке фото/видео - гарантируют качество

2) Что такое порождающий полином?

Порождающий полином - это специальное число, по которому считается контрольная сумма.

Простыми словами, это "правило расчета" CRC. Определяет, сколько битов будет в контрольной сумме

Каждый полином дает разные результаты

В нашем случае (вариант 12):

Полином: $x^7 + x^6 + x^4 + x^3 + x^2 + x$

В виде чисел: 11011110

Длина CRC: 7 бит (как 7-значный пароль)

3) Как вычислить CRC для пакета с данными?

На передаче:

Берем данные (например, 32 бита)

Проходим по каждому биту

Считаем формулу с полиномом

Получаем CRC (7 бит)

Отправляем: данные + CRC

На приеме:

Получаем данные + CRC

Снова считаем ту же формулу

Если получили 0000000 - ошибок нет, если получили другое - ошибка есть

Результат работы: В нашей программе CRC обнаруживает 100% одиночных ошибок при передаче данных.

4 Скрипты

Скрипт 1

c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
// Полином  $x^7 + x^6 + x^4 + x^3 + x^2 + x$ 
```

```
int poly[8] = {1,1,0,1,1,1,0};
```

```
// Функция вычисления CRC
```

```
void calculateCRC(int data[], int data_bits, int crc[]) {
```

```
    // Проходим по всем битам данных
```

```
    for(int i=0; i<data_bits; i++) {
```

```
        // Текущий бит данных
```

```
        int bit = data[i];
```

```
        // Старший (левый) бит CRC регистра
```

```
        int msb = crc[0];
```

```
        // Вычисляем обратную связь: XOR между старшим битом CRC и текущим битом данных
```

```
        int fb = msb ^ bit;
```

```
        // Сдвигаем CRC регистр влево на 1 бит
```

```
        for(int j=0; j<6; j++) crc[j] = crc[j+1];
```

```
        crc[6] = 0; // Младший бит заполняем нулем
```

```
        // Если обратная связь = 1, применяем полином через операцию XOR
```

```
        if(fb == 1) {
```

```
            for(int j=0; j<7; j++) crc[j] ^= poly[j+1];
```

```
        }
```

```
    }
```

```
}
```

```
// Функция проверки CRC
```

```
int checkCRC(int data[], int total_bits) {
```

```

int crc[7] = {0}; // Инициализируем CRC регистр нулями

// Проходим по всем битам (данные + CRC)
for(int i=0; i<total_bits; i++) {
    // Текущий бит из принятых данных
    int bit = data[i];

    // Старший бит CRC регистра
    int msb = crc[0];

    // Вычисляем обратную связь
    int fb = msb ^ bit;

    // Сдвигаем CRC регистр влево
    for(int j=0; j<6; j++) crc[j] = crc[j+1];
    crc[6] = 0;

    // Применяем полином если нужно
    if(fb == 1) {
        for(int j=0; j<7; j++) crc[j] ^= poly[j+1];
    }
}

// Проверяем результат: если все биты CRC = 0, ошибок нет
for(int i=0; i<7; i++) {
    if(crc[i] != 0) return 0; // Ошибка есть (ненулевой остаток)
}

return 1; // Ошибок нет (все биты нулевые)
}

int main() {
    srand(time(NULL)); // Инициализация генератора случайных чисел

    printf(" CRC проверка\n\n");
    printf("1. 32 бита:\n");

    // Фиксированные тестовые данные (32 бита)

```

```

int data32[32] = {1,0,1,1,0,0,1,0,1,1,0,0,1,1,0,1,0,0,0,1,0,0,1,0,0,0,1,1,0,1,0,0};
int crc32[7] = {0}; // CRC регистр (7 бит)

// Вычисляем CRC для исходных данных
calculateCRC(data32, 32, crc32);

// Выводим исходные данные
printf("Данные: ");
for(int i=0; i<32; i++) printf("%d", data32[i]);

// Выводим вычисленный CRC
printf("\nCRC: ");
for(int i=0; i<7; i++) printf("%d", crc32[i]);

// Создаем массив данных с добавленным CRC
int data_with_crc32[39]; // 32 данных + 7 CRC = 39 бит
for(int i=0; i<32; i++) data_with_crc32[i] = data32[i]; // Копируем данные
for(int i=0; i<7; i++) data_with_crc32[32+i] = crc32[i]; // Добавляем CRC

// Проверяем корректность передачи (без ошибок)
printf("\nПроверка: %s\n\n", checkCRC(data_with_crc32, 39) ? "Ошибок нет" : "Ошибка!");

printf("2. 250 бит:\n");

// Генерируем случайные данные (250 бит)
int data250[250];
for(int i=0; i<250; i++) data250[i] = rand() % 2; // 0 или 1

int crc250[7] = {0}; // CRC регистр

// Вычисляем CRC для 250 бит
calculateCRC(data250, 250, crc250);

```

```

// Выводим вычисленный CRC
printf("CRC: ");
for(int i=0; i<7; i++) printf("%d", crc250[i]);

// Создаем данные с CRC (250 + 7 = 257 бит)
int data_with_crc250[257];
for(int i=0; i<250; i++) data_with_crc250[i] = data250[i]; // Данные
for(int i=0; i<7; i++) data_with_crc250[250+i] = crc250[i]; // CRC

// Тестируем обнаружение ошибок
int good = 0, bad = 0; // Счетчики обнаруженных и пропущенных ошибок

// Тестируем все возможные позиции одиночных ошибок
for(int i=0; i<257; i++) {
    int test[257]; // Тестовый массив

    // Копируем исходные данные с CRC
    for(int j=0; j<257; j++) test[j] = data_with_crc250[j];

    // Добавляем ошибку: инвертируем i-й бит
    test[i] = 1 - test[i];

    if(checkCRC(test, 257)) {
        bad++; // Ошибка НЕ обнаружена
    } else {
        good++; // Ошибка обнаружена
    }
}

printf("\nОбнаружено ошибок: %d/%d (%.1f%%)\n", good, 257, (float)good/257*100);
return 0;

```

}